

Sofia Lima dos Santos

Paradigmas da Programação - Lista 3

Exercício 1:

Em linguagem de programação Java, uma interface é um tipo de referência, similar a uma classe, que pode conter apenas constantes, assinaturas de métodos, métodos padrão, métodos estáticos e tipos aninhados. Corpos de métodos existem apenas para métodos padrão e métodos estáticos. Interfaces não podem ser instanciadas – elas só podem ser implementadas por classes ou estendidas por outras interfaces.

Referência: (<https://docs.oracle.com/javase/tutorial/java/landl/createinterface.html>)

O exemplo abaixo cria uma interface Animal com os métodos andar, comer e correr.

A classe Gato implementa a interface Animal, ou seja, a classe estabeleceu um "contrato" com a interface, e agora deve definir todos os métodos declarados em Animal.

Interface Animal

```
package exerc01;
public interface Animal {
    void andar();
    void comer();
    void correr();
}
```

Classe Gato

```
package exerc01;
public class Gato implements Animal{
    @Override
    public void andar() {
        System.out.print("Andando...");
    }

    @Override
    public void comer() {
        System.out.print("Comendo...");
    }

    @Override
    public void correr() {
        System.out.print("Correndo...");
    }
}
```

Exercício 2:

Senha.java

```
package exerc02;

public class Senha {
    private String senha;
    private int tentativas = 0;
    private boolean bloq = false;

    public Senha(String senhaInicial) {
        this.senha = senhaInicial;
    }

    private boolean verificaSenha(String s) {
        return senha.equals(s);
    }

    public boolean estaBloqueada() {
        return bloq;
    }

    public boolean entraSenha(String s) {
        if (bloq) {
            System.out.println("A senha está bloqueada.");
            return false;
        }

        if (verificaSenha(s)) {
            System.out.println("Senha correta.");
            tentativas = 0;
            return true;
        } else {
            tentativas++;
            if (tentativas < 3) {
                System.out.println("Senha incorreta. Restam " + (3 - tentativas) + " tentativas.");
            } else {
                System.out.println("Senha incorreta. Você atingiu o limite de tentativas.");
                bloq = true;
            }
            return false;
        }
    }
}
```

```

    public void trocaSenha(String senhaAntiga, String senhaNova) {
        if (estaBloqueada()) {
            System.out.println("A senha está bloqueada, não é possível
alterá-la.");
        } else {
            if (entraSenha(senhaAntiga)) {
                senha = senhaNova;
                tentativas = 0;
                System.out.println("Senha alterada com sucesso.");
            } else{
                System.out.println("Senha antiga incorreta. Não foi
possível alterar a senha.");
                tentativas++;
                if (tentativas >= 3) {
                    bloq = true;
                    System.out.println("Você atingiu o limite de tentativas
e a senha foi bloqueada.");
                }
            }
        }
    }
}

```

TestaSenha:

```

package exerc02;
import java.util.Scanner;

public class TestaSenha {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Defina a senha: ");
        String senhaInicial = scanner.nextLine();
        Senha senha = new Senha(senhaInicial);

        boolean continuar = true;

        while (continuar) {
            System.out.println("\nMenu:");
            System.out.println("1. Entrar com senha");
            System.out.println("2. Trocar senha");
            System.out.println("3. Verificar se a senha está bloqueada");
            System.out.println("4. Sair");
            System.out.print("Escolha uma opção: ");

```

```

        int opcao = scanner.nextInt();
        scanner.nextLine(); // Consome a nova linha

        switch (opcao) {
            case 1:
                System.out.print("Digite a senha: ");
                String tentativaSenha = scanner.nextLine();
                senha.entraSenha(tentativaSenha);
                break;
            case 2:
                System.out.print("Digite a senha antiga: ");
                String senhaAntiga = scanner.nextLine();
                System.out.print("Digite a nova senha: ");
                String senhaNova = scanner.nextLine();
                senha.trocaSenha(senhaAntiga, senhaNova);
                break;
            case 3:
                System.out.println("A senha está bloqueada? " +
senha.estaBloqueada());
                break;
            case 4:
                continuar = false;
                break;
            default:
                System.out.println("Opção inválida! Tente novamente.");
        }
    }

    scanner.close();
}
}

```

Exercício 3:

Porta.java

```

package exerc03;

public class Porta {
    private boolean aberta;
    private String cor;
    private double dimensaoX, dimensaoY, dimensaoZ;
}

```

```

    public Porta(String _cor, boolean _aberta, double _dimensaoX, double
_dimensaoY, double _dimensaoZ) {
        this.cor = _cor;
        this.aberta = _aberta;
        this.dimensaoX = _dimensaoX;
        this.dimensaoY = _dimensaoY;
        this.dimensaoZ = _dimensaoZ;
    }

    public void abre() {
        this.aberta = true;
    }
    public void fecha(){
        this.aberta = false;
    }

    public void pinta(String cor){
        this.cor = cor;
    }

    public boolean estaAberta() {
        return this.aberta;
    }
}

```

Exercício 4:

Casa.java

```

package exerc04;
import exerc03.Porta;

public class Casa {
    private Porta porta1;
    private Porta porta2;
    private Porta porta3;
    private String cor;

    public Casa(Porta porta1, Porta porta2, Porta porta3, String cor) {
        this.porta1 = porta1;
        this.porta2 = porta2;
        this.porta3 = porta3;
        this.cor = cor;
    }
}

```

```

public void pinta(String cor){
    this.cor = cor;
}

public int quantasPortasEstaoAbertas(){
    int c = 0;
    if (porta1.estaAberta()) {
        c++;
    }
    if (porta2.estaAberta()) {
        c++;
    }
    if (porta3.estaAberta()) {
        c++;
    }
    return c;
}

public String getCor(){
    return cor;
}

public Porta getPorta1(){
    return porta1;
}

public Porta getPorta2(){
    return porta2;
}

public Porta getPorta3(){
    return porta3;
}
}

```

TestaCasa

```

package exerc04;
import exerc03.Porta;

public class TestaCasa {
    public static void main(String[] args) {
        Porta porta1 = new Porta("Marrom", false, 0.76, 2.10, 0.74);
        Porta porta2 = new Porta("Branca", true, 0.78, 2.00, 0.71);
        Porta porta3 = new Porta("Preta", false, 0.77, 1.90, 0.70);
    }
}

```

```

        Casa casa = new Casa(porta1, porta2, porta3,"Azul");

        System.out.println("A cor da casa é: " + casa.getCor());
        casa.pinta("Laranja");
        System.out.println("A nova cor da casa é: " + casa.getCor());

        System.out.println("A quantidade de portas abertas é: " +
casa.quantasPortasEstaoAbertas());

        casa.getPorta1();
        casa.getPorta2();
        casa.getPorta3();
    }
}

```

Exercício 5:

Calculadora.java

```

package exerc05;

public class Calculadora {

    public double somar(double a, double b) {
        return a + b;
    }

    public double subtrair(double a, double b){
        return a - b;
    }

    public double multiplicar(double a, double b) {
        return a * b;
    }

    public double dividir(double a, double b){
        return a / b;
    }

}

```

CalculadoraCientifica

```

package exerc05;

public class CalculadoraCientifica extends Calculadora {

```

```

public double potencia(double b, double e) {
    return Math.pow(b, e);
}
/*
Tentei fazer dessa forma também para usar o método multiplicar
já que era pra usar herança mas aí "e" deveria ser int

double resultado = 1.0;
for (int i = 0; i < e; i++) {
    resultado = multiplicar(resultado, b);
}
*/
}

```

Exercício 6:

Data.java

```

package exerc06;
public class Data {
    private int dia;
    private int mes;
    private int ano;

    public Data(int _dia, int _mes, int _ano) {
        this.dia = _dia;
        this.mes = _mes;
        this.ano = _ano;
    }

    int[] diasMes = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    public boolean dataValida() {
        if (mes < 1 || mes > 12 || dia < 1 || ano < 1) {
            return false;
        } else if (mes == 2 && anoBissexto() && dia <= 29) {
            return true;
        } else if (dia > diasMes[mes]) {
            return false;
        }
        return true;
    }
}

```



```

private boolean anoBissexto() {
    if ((ano % 4 == 0 && ano % 100 != 0) || (ano % 400 == 0)) {
        return true;
    } else {
        return false;
    }
}

public void imprimirData() {
    if (!dataValida()) {
        System.out.println("Data inválida");
    } else {
        System.out.println(dia + "/" + mes + "/" + ano);
    }
}

public void proximoDia() {
    if (dataValida()) {
        dia += 1;

        if ((mes == 2 && dia > 28 && !anoBissexto()) || (mes == 2 &&
dia > 29 && anoBissexto()) || (dia > diasMes[mes])) {
            dia = 1;
            mes += 1;

            if (mes > 12) {
                mes = 1;
                ano += 1;
            }
        }
        imprimirData();
    }
}
}

```

TestaData

```

package exerc06;
import java.util.Scanner;

public class TestaData {
    public static void main(String[] args){
        Scanner scanner = new Scanner(System.in);

        System.out.print("Digite o dia: ");
    }
}

```

```

    int dia = scanner.nextInt();

    System.out.print("Digite o mês: ");
    int mes = scanner.nextInt();

    System.out.print("Digite o ano: ");
    int ano = scanner.nextInt();

    Data d = new Data(dia, mes, ano);

    d.imprimirData();
    d.proximoDia();
}
}

```

Exercício 7:

a) O valor mostrado na tela será 10. A variável lamp2 é atribuída com a mesma referência que lamp1, assim, a condição estabelecida é verdadeira e o bloco if é executado, porém como o incremento só acontece depois que o valor de i foi avaliado, é exibido o valor original, que é 10.

b) Nada é mostrado na tela, porque a condição de igualdade não é atendida, já que lamp1 e lamp2 são instâncias diferentes da classe Lampada.

c) O valor é 11.

d) O valor é 10.

e) No código da esquerda: já que lamp1 e lamp2 possuem a mesma referência, ao alterar o tipo da lamp1 para “halógena”, o tipo de lamp2 também passará a ser “halógena”.

No código da direita: A mudança de tipo de lamp1 não afetará lamp2 por serem instâncias diferentes, logo, quando lamp2.mostrarInformaçõesGerais() for chamada, o tipo de lamp2 ainda será “LED”.